

Product Adoption & Value Realization

Is what customers bought actually being used —
and what is the gap worth?

I build the layer that tells a business what's actually happening

Data science → data platform → streaming infrastructure. Ten years shipping the measurement layer other teams run on — Mu Sigma, Bing Ads, Azure Event Hubs, Microsoft 365 Data Platform.

1

You can't fix what you can't see

Built unified telemetry across Azure Event Hubs and Service Bus — surfaced \$200K/month in billing discrepancies nobody knew existed.

2

Choosing the metric is a product decision

Teams Queues App: prioritized the metrics supervisors needed over the ones that were easy to instrument. 25× adoption in six months.

3

Trust is a feature

Self-service governance across 200+ service owners — 100% EDPS compliance, 40 weeks of manual effort removed each year.

Three questions every software business answers precisely. And one it can't.

What do we sell?

Every SKU, every feature, catalogued



How much did we sell?

To the dollar, to the unit



Who did we sell it to?

Segment, region, industry, contract terms



What are they doing with it?

No shared definition, no shared number



The fourth question is the only one that tells you whether the first three will still be true next year.

Revenue tells you what a customer bought. Nothing tells you whether it mattered.

More than half of what customers buy never converts into use — and nobody sees it until the renewal call.

By then the only lever left is a discount. I'll show you exactly how I get to that number — and you should hold me to it.

One score, and it's two things multiplied

$$\text{VRR} = \% \text{ of their features they use} \times \% \text{ of their capacity they use}$$

Acme Financial • \$100k/year • 4 features • 10M credits • measured monthly

Features used	1 of 4, value-weighted	33%
Capacity used	667k of 833k credits this month	80%
Value Realization Rate	33% × 80%	26%

They bought a platform and deployed a point tool.

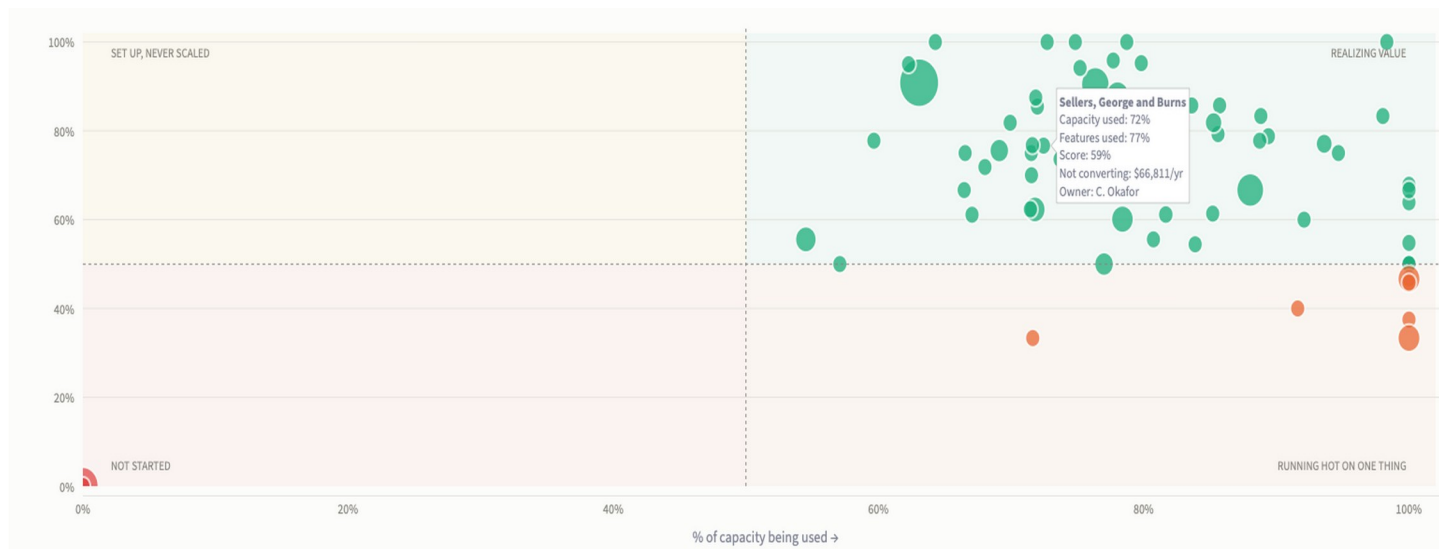
Neither number catches this alone. Consumption looks healthy. A feature checklist looks half-done.

Capped at 100% — over-consumption is an expansion signal, not a health signal. • Features are value-weighted: Core 3×, Differentiator 2×, Adjacent 1×.

An earlier version used a geometric mean of three factors. More precise, unexplainable in a room — so explainability became a design requirement, not a preference.

The same score, four different problems

A single number tells you an account is at 26%. The two factors tell you which of four conversations to have — and who should have it.



Not started

12 accounts • \$3.9M

Deployment rescue • CS leadership

Running hot on one thing

7 accounts • \$1.5M

They will hit the cap and renegotiate on price • Sales + Product

Set up, never scaled

thin at this threshold

Enablement — why hasn't volume grown? • Customer Success

Realizing value

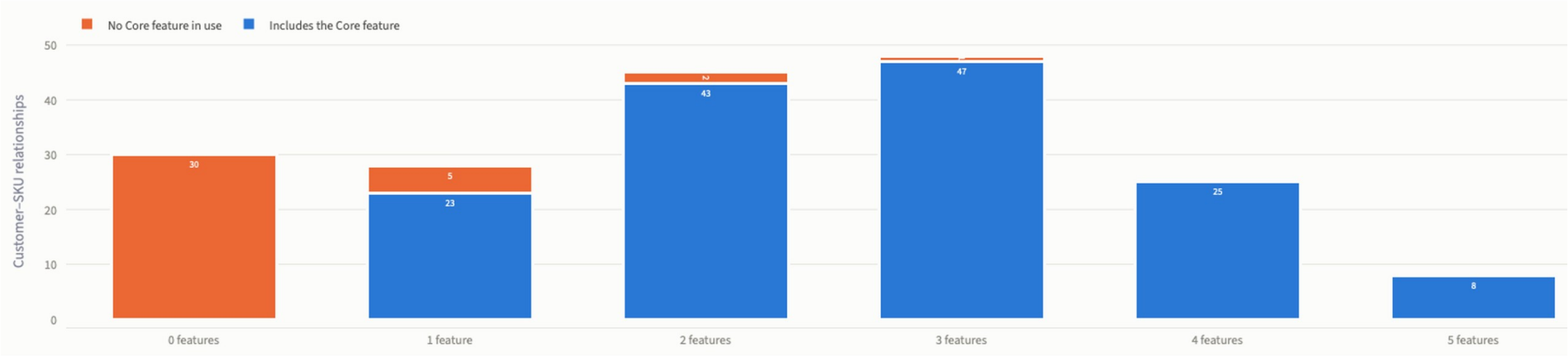
64 accounts • \$7.8M

Expansion conversation • Sales

Bottom-left is a rescue. Bottom-right is a pricing conversation waiting to happen. Top-left is an enablement gap. Top-right is where expansion comes from.

Live in the dashboard, and the threshold is a slider — so “high” is a dial, not a decree. Bubble size is contract value; hovering gives the account, its score, and what it is worth.

What it found — and the number that should worry you



4% use every feature they bought

32% use one feature or none

21% use no Core feature at all

Which is where the number from earlier comes from: **\$13.2M of \$25M** in active contract value is not converting.

Blue means the Core feature is among those in use; orange means it is not. Counted per customer-SKU relationship, not per customer.

LIVE

Six minutes in the dashboard

Everything here is read from the pipeline. Nothing is computed in the app — the dashboard and the tests are looking at the same numbers.

- 1 Open on the Overview**

The trust line first: data complete through July, three internal accounts excluded, eighty-four duplicate rows removed.
- 2 Move the threshold slider**

Every account, plotted. Drag 50% to 70% and watch them redistribute. “High” is a dial, not a decree.
- 3 Drill into one account**

Thirteen named features they are paying for and not using — three of them the reason they bought it. This is what a CSM opens.
- 4 Open Data Quality**

Every row this pipeline dropped, with a reason and a count, appended on every run.

Move 3 is the one that matters — it is where a metric becomes a phone call.

One metric, three altitudes, one accountable owner

CPO & GMs

Where to invest, what to repackage

Portfolio score and dollars not converting, cut by segment

Product

Which SKU is failing, and in which of the four ways

Feature-level adoption inside each SKU

CSM & Sales

Which account to call this week

The exact unused features and what they are worth

Every flag ends in a worklist, not a score

What a CSM actually opens:

“Acme Financial is paying for 13 features they did not use this month — **3 of them are the reason the SKU was bought**. Roughly \$1.8M a year of contract value sits behind this list.”

Then the named features, ranked by how much they matter — Core first.

Not “this account scores 26%.”

And it becomes accountable.

Value at Risk = contract value $\times$ (1 – VRR). It is in dollars, so it sums by account owner. Quarter over quarter you can see how much at-risk value each owner moved — the bridge from a metric to a motion.

THE ASK

Align on Value Realization Rate as how we measure adoption.

One definition. One calculation. Every team reading the same number.



Now

Agree the metric and the calculation. Everything downstream depends on one number we all accept.



Next quarter

Backtest against real renewal history. Does it predict churn earlier than NRR?



After that

Derive feature weights from outcomes instead of judgment — then, and only then, the incentive conversation.

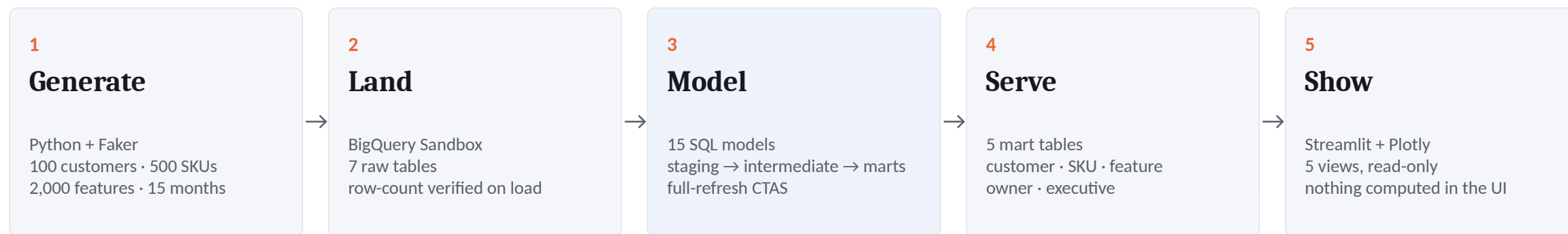
APPENDIX

Technical detail, for the questions

Architecture and trade-offs · the twelve messy realities · how I know the number is right · what I got wrong

The recommendation ends on the previous slide. Everything past here exists to be challenged.

How the data flows



Four decisions worth defending

Full-refresh CTAS, not incremental

The sandbox blocks DML — I verified that with a smoke test rather than reading it in docs. It is also the right call at this volume: idempotent, no state drift.

Ratios diagnose, dollars prioritize

Percentages cannot be summed across customers. Dollars can, so Value at Risk aggregates to any cut of the business.

Streamlit, not React

A deliberate speed trade-off. The deliverable is a decision, not a product.

Nothing hardcoded

Project, dataset, and thresholds all come from config. The pipeline runs against any BigQuery instance by changing one file.

The hard part: the data is a liar

Twelve realities injected on purpose. Five were specified. Seven I added, because they are what actually breaks adoption metrics in production.

CUSTOMER BEHAVIOUR • 5, SPECIFIED

Shelfware • burn-and-stop • chronic overage • mid-year expansion • brand-new deployments

DATA INTEGRITY • 7, ADDED

Duplicate rows • unlimited contracts • usage on unbought features • lapsed renewals • late-arriving data • internal test accounts • multi-currency

Three of them changed the framework, not just the cleaning rules

Some contracts have no denominator

“Unlimited” deals are scored on feature coverage alone. Substituting 100% would make them look like the healthiest accounts in the portfolio.

There is a third state

A lapsed renewal is not healthy and not at risk — it is not applicable, and produces no row at all. Most frameworks only have two states.

A number can be correct but not final

The current month is marked incomplete and drawn greyed. Hide that, and the first time an exec notices it move, they stop trusting every number.

Row counts prove it ran. These prove it was right.

Ground truth the pipeline cannot see

The generator secretly labels each customer's intended behaviour in a column no model reads. The tests ask whether the metric independently rediscovered what was planted.

40 automated tests

Structure, injection, business-answer correctness, and invariants — including one that simply asserts the portfolio value is plausible.

Every dropped row is on the record

Each cleaning step logs rows in, rows out, and why. Appended per run, so a number that moves can be traced to the step that moved it.

What the last run actually did

- 84** duplicate contract-months removed
- 3** internal test accounts excluded
- 395** usage rows for features never bought
- 684** months with a contract and no usage — counted as zero, not unknown
- 144** rows in the current month marked still loading
- 4** months with no active contract — not scored, not shelfware

Every row, every run, with a reason — and shown in the dashboard.

Silent cleaning is how data teams lose trust.

A number moves, nobody can explain it, and after that every number gets questioned. So the dashboard carries a permanent line above the metrics: what was excluded, what was deduplicated, and how current the data is.

What went wrong, and what it changed

I read past a wrong number.

An early version reported \$1.9B at risk across 97 customers — larger than the business. I had multiplied consumption capacity by unit price instead of the commercial quantity. The pipeline reported success throughout.

There is now a test that asserts the portfolio value is plausible. Structure was never the problem; magnitude was.

My own safety mechanism was never exercised.

The audit's first run showed zero rows zero-filled. My generator was emitting zeros where real telemetry emits nothing, so the anti-shelfware logic had never run and its tests were passing vacuously.

The audit caught it, not the tests. That is the argument for having built it.

Synthetic data validates mechanics, not predictive power.

These tests prove the logic handles twelve messy realities. They cannot prove VRR predicts churn.

That needs a backtest against real history — which is exactly what I am asking for.